Bases de données relationnelles(MYSQL)

-une base de données **relationnelle** comme **mysql** organise les informations sous forme de **tables** reliées entre elles par des **relations**. Cela permet une structure rigide et un contrôle strict de l'intégrité des données.

-les bases de données **non relationnelles** comme **MongoDB** utilisent des structures plus flexibles, comme des documents en format **JSON**, pour stocker les informations, offrant ainsi plus de souplesse dans certains types d'applications.

1. Connexion et gestion des bases de données dans MySQL

- **Connexion à MySQL** :

```
mysql -u root -p
```

- **Afficher les bases de données existantes** :

```
SHOW DATABASES;
```

- **Créer une nouvelle base de données** :

```
CREATE DATABASE nom_de_la_base;
```

- **Utiliser une base de données** :

```
USE nom_de_la_base;
```

2. Gestion des tables dans MySQL (DDL)

Les tables constituent l'élément fondamental des bases de données relationnelles, et chaque table contient des lignes et des colonnes.

- **Créer une table** :
  - Exemple :

```
CREATE TABLE utilisateurs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(50),
  email VARCHAR(100),
  age INT
);
```

- **Afficher la structure d'une table** :
  - Permet d'afficher les colonnes d'une table, ainsi que leurs types de données.
  - Exemple :

```
DESC utilisateurs;
```

- **Modifier une table (ALTER)** :
  - **Ajouter une colonne** :
    - Exemple :

```
ALTER TABLE utilisateurs ADD adresse VARCHAR(255);
```

  - **Changer le type d'une colonne** :
    - Exemple :

```
ALTER TABLE utilisateurs CHANGE age age INT(3);
```

  - **Supprimer une colonne** :
    - Exemple :

```
ALTER TABLE utilisateurs DROP COLUMN adresse;
```

- **Ajouter une contrainte (constraint)** :
  - Une contrainte comme une **clé étrangère** permet de créer des liens entre différentes tables afin de garantir l'intégrité des données.
  - Exemple :

```
ALTER TABLE commandes ADD CONSTRAINT fk_utilisateur
FOREIGN KEY (utilisateur_id) REFERENCES utilisateurs(id);
```

- **Ajouter une contrainte CHECK**

  - Exemple :

    Pour ajouter une contrainte qui vérifie que l'âge est d'au moins 18 ans, nous utiliserons la commande suivante :

    ```
    ALTER TABLE utilisateurs
    ADD CONSTRAINT check_age CHECK (age >= 18);
    ```

- **Ajouter une contrainte CHECK avec REGEXP**

  - Exemple :

Ajouter une contrainte qui vérifie que l'email respecte un format spécifique, en utilisant une expression régulière. Dans cet exemple, nous vérifierons que l'email a la forme standard d'une adresse email (par exemple, `nom@domaine.com`).

```
ALTER TABLE utilisateurs
ADD CONSTRAINT check_email CHECK (email REGEXP '^[A-Za-z0-9._%+-
]+@[A-Za-z0-9.-]+\\.[A-Z|a-z]{2,}$');
```

3. Manipulation des données (DML)

Les **DML (Data Manipulation Language)** sont les commandes permettant de manipuler directement les données contenues dans les tables.

- **Insérer des données (INSERT)** :
  - Ajoute une nouvelle ligne dans une table.
  - Exemple :

    ```
    INSERT INTO utilisateurs (nom, email, age) VALUES
    ('Alice', 'alice@example.com', 30);
    ```

- **Mettre à jour des données (UPDATE)** :
  - Modifie des valeurs existantes dans une ou plusieurs colonnes d'une table.
  - Exemple :

    ```
    UPDATE utilisateurs SET age = 31 WHERE nom = 'Alice';
    ```

- **Supprimer des données (DELETE)** :
  - Supprime des lignes d'une table.
  - Exemple :

    ```
    DELETE FROM utilisateurs WHERE nom = 'Alice';
    ```

---

4. Requêtes de sélection(DIL) (SELECT)

La commande **SELECT** permet de récupérer des données d'une ou plusieurs tables.

- **Sélectionner toutes les données d'une table** :
  - Exemple :

```
SELECT * FROM utilisateurs;
```

- **Sélectionner des colonnes spécifiques** :
  - Exemple :

```
SELECT nom, email FROM utilisateurs;
```

- **Filtrer avec une condition (WHERE)** :
  - Exemple :

```
SELECT * FROM utilisateurs WHERE age > 25;
```

5. Les jointures :combinaisons

5-1. Requête de combinaison implicite :

La requête de combinaison implicite est une méthode traditionnelle utilisée pour relier des données de plusieurs tables en SQL. Dans cette approche, vous spécifiez les tables dans la clause `FROM`, puis vous utilisez la clause `WHERE` pour définir les conditions qui relient les tables.

  - Exemple

Considérons deux tables : `utilisateurs` et `commandes`.

Tables :

- utilisateurs :

id	nom	age
1	Alice	30
2	Bob	25
3	Charlie	35

- commandes :

id	utilisateur_id	produit
1	1	Produit A
2	2	Produit B
3	1	Produit C

- Requête

```
SELECT T.*, T2.* FROM utilisateurs T, commandes T2
WHERE T.id = T2.utilisateur_id;
```

id	nom	age	id	utilisateur_id	produit
1	Alice	30	1	1	Produit A
1	Alice	30	3	1	Produit C
2	Bob	25	2	2	Produit B

5-2. Requête de jointure explicite

La requête de jointure explicite utilise la syntaxe `JOIN` pour relier les tables. Cela permet de définir clairement le type de jointure que vous souhaitez effectuer (par exemple, `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, etc.) et rend les requêtes plus lisibles et maintenables.

Voici des exemples illustrant les différents types de jointures explicites (INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN) en utilisant les tables **utilisateurs** et **commandes**.

utilisateurs

id	nom
1	Alice
2	Bob
3	Charlie

commandes

id	utilisateur_id	produit
1	1	Produit A
2	1	Produit B
3	2	Produit C
4	4	Produit D

5-2-1. **INNER JOIN**

```
SELECT U.nom, C.produit FROM utilisateurs AS U INNER JOIN commandes
AS C ON U.id = C.utilisateur_id;
```

Résultat :

nom	produit
Alice	Produit A
Alice	Produit B
Bob	Produit C

Dans ce cas, seules les lignes des utilisateurs ayant des commandes sont récupérées. Charlie n'apparaît pas car il n'a pas de commandes.

5-2-2. **LEFT JOIN**

Requête :

```
SELECT U.nom, C.produit
FROM utilisateurs AS U
LEFT JOIN commandes AS C ON U.id = C.utilisateur_id;
```

Résultat :

nom	produit
Alice	Produit A
Alice	Produit B
Bob	Produit C
Charlie	NULL

Cette jointure récupère toutes les lignes de la table utilisateurs, même celles sans correspondance dans commandes. Charlie apparaît avec NULL pour le produit car il n'a pas passé de commande.

5-2-3. **RIGHT JOIN**

Requête :

```
SELECT U.nom, C.produit FROM utilisateurs AS U
RIGHT JOIN commandes AS C ON U.id = C.utilisateur_id;
```

Résultat :

nom	produit
Alice	Produit A
Alice	Produit B
Bob	Produit C
NULL	Produit D

Cette jointure récupère toutes les lignes de la table commandes, même celles sans correspondance dans utilisateurs. Dans ce cas, Produit D n'a pas d'utilisateur associé car utilisateur_id est 4, qui n'existe pas dans la table utilisateurs.

5-2-4. **FULL JOIN**

Requête :

```
SELECT U.nom, C.produit FROM utilisateurs AS U
FULL JOIN commandes AS C ON U.id = C.utilisateur_id;
```

Résultat :

nom	produit
Alice	Produit A
Alice	Produit B
Bob	Produit C
Charlie	NULL
NULL	Produit D

Cette jointure récupère toutes les lignes des deux tables. Charlie apparaît avec NULL pour le produit car il n'a pas de commandes, et Produit D apparaît avec NULL pour le nom car son utilisateur_id n'existe pas dans la table utilisateurs.

Dans MySQL, la clause **FULL JOIN** n'est pas prise en charge nativement. Cependant, vous pouvez obtenir un résultat équivalent en combinant un **LEFT JOIN** et un **RIGHT JOIN** avec une union des résultats.

Voici comment réécrire votre requête pour simuler un **FULL OUTER JOIN** dans MySQL :

SELECT U.nom, C.produit

FROM utilisateurs AS U

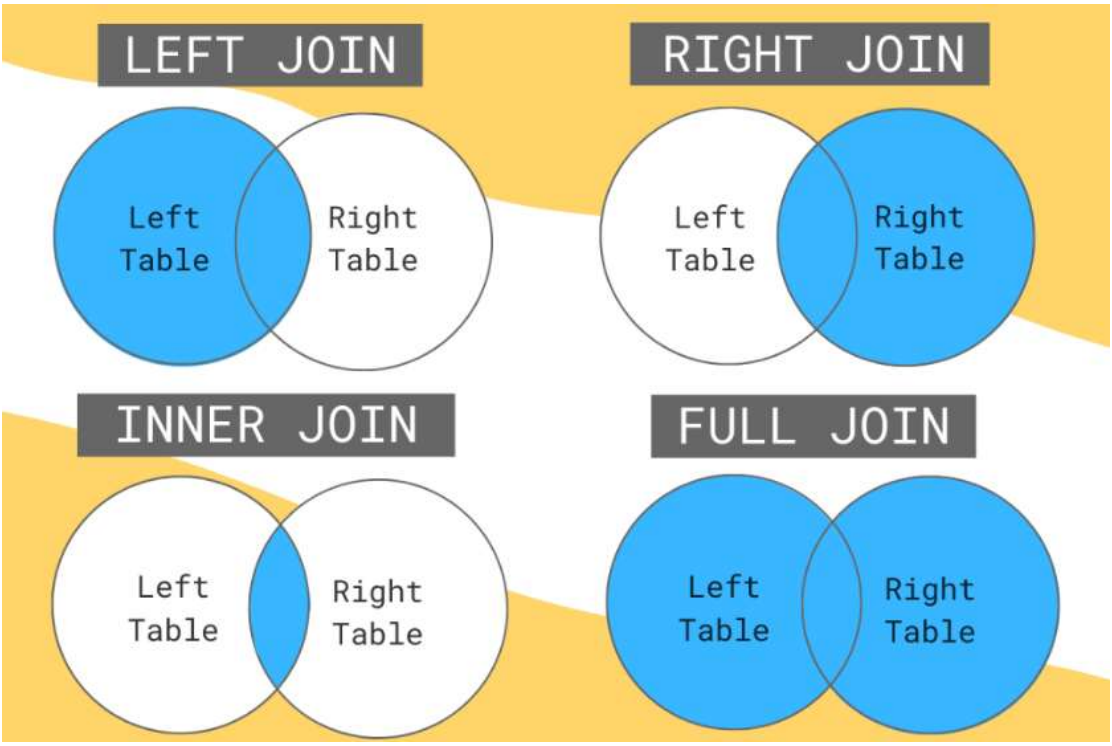
LEFT JOIN commandes AS C ON U.id = C.utilisateur_id

UNION

SELECT U.nom, C.produit

FROM utilisateurs AS U

RIGHT JOIN commandes AS C ON U.id = C.utilisateur_id;


LEFT JOIN
Left Table
Right Table
RIGHT JOIN
Left Table
Right Table
INNER JOIN
Left Table
Right Table
FULL JOIN
Left Table
Right Table

6-Opérateurs ensemblistes :

AviondeAF

immat	typeAvion	nbHVol
F-WTSS	Concorde	6570
F-GLFS	A320	3500
F-GTMP	A340	

AviondeSING

immatriculation	typeAv	PrixAchat
S-ANSI	A320	104 500
S-AVEZ	A320	156 000
S-SMILE	A330	198 000
F-GTMP	A340	204 500

Une des forces du modèle relationnel repose sur le fait qu'il est fondé sur une base mathématique (théorie des ensembles). Le langage SQL devrait programmer les opérations binaires (entre deux tables -requettes) suivantes :

- **intersection** qui extrait des données présentes simultanément dans les deux tables(requettes) ;
- **union** par les opérateurs `UNION` et `UNION ALL` qui fusionnent des données des deux tables(requettes) ;
- **différence** qui extrait des données présentes dans une table sans être présentes dans la deuxième table ;

6-1.Intersection :

L'intersection entre deux ensembles homogènes se programme à l'aide d'une requête du type `SELECT DISTINCT` *ensemble1* `FROM` *Table1* `WHERE` *ensemble1* `IN (SELECT` *ensemble2* `FROM` *Table2*). Comme l'opérateur d'intersection, cette requête est commutative (*Table1* peut jouer le rôle de *Table2*, et *ensemble1* celui d'*ensemble2*).

Besoin	Requête
Quels sont les types d'avions que les deux compagnies exploitent en commun ?	`SELECT DISTINCT typeAvion FROM AvionsdeAF WHERE typeAvion IN (SELECT typeAv FROM AvionsdeSING);`

```
+------------+
| typeAvion  |
+------------+
| A320       |
| A340       |
+------------+
```

Besoin	Requête
Quels sont les avions qui sont exploités par les deux compagnies en commun ?	`SELECT DISTINCT immatriculation ,typeAv FROM AvionsdeSING WHERE immatriculation IN (SELECT immat FROM AvionsdeAF) AND typeAv IN (SELECT typeAvion FROM AvionsdeAF);`

```
+-----------------+---------+
| immatriculation | typeAv  |
+-----------------+---------+
| F-GTMP          | A340    |
+-----------------+---------+
```

6-2.Opérateurs UNION et UNION ALL :

Les opérateurs `UNION` et `UNION ALL` sont commutatifs. L'opérateur `UNION` permet d'éviter les duplicatas (comme `DISTINCT` dans un `SELECT`). `UNION ALL` ne les élimine pas.

Besoin	Requête

Quels sont tous les types d'avions que les deux compagnies exploitent ?	`SELECT typeAvion FROM AvionsdeAF`
	`UNION`
	`SELECT typeAv FROM AvionsdeSING;`
	`+-----------+`
	`| typeAvion |`
	`+-----------+`
	`| A320 |`
	`| A340 |`
	`| Concorde |`
	`| A330 |`
	`+-----------+`

Besoin	Requête

Même requête avec les duplicatas. On extrait les types de la compagnie 'AF', suivis des types de la compagnie 'SING'.	`SELECT typeAvion FROM AvionsdeAF`
	`UNION ALL`
	`SELECT typeAv FROM AvionsdeSING;`
	`+-----------+`
	`| typeAvion |`
	`+-----------+`
	`| A320 |`
	`| A340 |`
	`| Concorde |`
	`| A340 |`
	`| A320 |`
	`| A320 |`
	`| A330 |`
	`+-----------+`

Requête	Résultat

`SELECT immatriculation,`
`  1.2*prixAchat` **`px`** `FROM Avions-deSING`
`UNION`
`  SELECT immat, 0 FROM AvionsdeAF`
`ORDER BY` **`px`** `DESC;`
	`+-----------------+------------+`
	`| immatriculation | px |`
	`+-----------------+------------+`
	`| F-GTMP | 245400.000 |`
	`| S-MILE | 237600.000 |`
	`| S-AVEZ | 187200.000 |`
	`| S-ANSI | 125400.000 |`
	`| F-GLFS | 0.000 |`
	`| F-GTMP | 0.000 |`
	`| F-WTSS | 0.000 |`
	`+-----------------+------------+`

6-3.Différence :

La différence entre deux ensembles homogènes se programme à l'aide d'une requête du type `SELECT DISTINCT` *ensemble1* `FROM` *Table1* `WHERE` *ensemble1* `NOT IN (SELECT` *ensemble2*
`FROM` *Table2*). Comme l'opérateur ensembliste, cette requête n'est pas commutative, car elle programme *ensemble1-ensemble2*.

Besoin	Requête

Quels sont les types d'avions exploités par la compagnie 'AF', mais pas par 'SING' ?	`SELECT DISTINCT typeAvion FROM AvionsdeAF`
	`  WHERE typeAvion NOT IN`
	`  (SELECT typeAv FROM AvionsdeSING);`
	`+-----------+`
	`| typeAvion |`
	`+-----------+`
	`| Concorde  |`
	`+-----------+`

Quels sont les types d'avions exploités par la compagnie 'SING,' mais pas par 'AF' ?	`SELECT DISTINCT typeAv FROM AvionsdeSING`
	`  WHERE typeAv NOT IN`
	`  (SELECT typeAvion  FROM AvionsdeAF);`
	`+--------+`
	`| typeAv |`
	`+--------+`
	`| A330   |`
	`+--------+`

6-4.Utilisation de « ANY / ALL » :

Le mot-clé ALL spécifie tous les éléments retournés tandis que le mot-clé ANY spécifie l'un d'entre eux.

Syntaxe:

> ALL (1,2,3) est équivalent à > 3

> ANY (1,2,3) est équivalent à > 1

6-5.Utilisation de « EXISTS » :

Le mot-clé EXISTS (et NOT EXISTS)

🔹 **C'est quoi ?**

EXISTS sert à vérifier si quelque chose existe dans une autre table.
Autrement dit, on s'en sert pour savoir si une sous-requête trouve des résultats ou non.

- **Si la sous-requête trouve au moins une ligne, alors EXISTS = VRAI ✅**
- **Si la sous-requête ne trouve rien, alors EXISTS = FAUX ❌**

Quand on écrit NOT EXISTS, c'est l'inverse :

- **NOT EXISTS = VRAI quand la sous-requête ne renvoie aucune ligne.**

🔹 **Exemple simple**

Imaginons deux tables :

Table Client

ClientId	Nom
1	Alice
2	Bob
3	Charlie

Table Commande

CommandeId	ClientId	Produit
1	1	Produit A
2	2	Produit B

🔸 **1. Trouver les clients qui n'ont pas passé de commande**

```
SELECT *

FROM Client cl

WHERE NOT EXISTS (

   SELECT *

   FROM Commande co

   WHERE co.ClientId = cl.ClientId

);
```

Comment ça marche ?

- **Pour chaque client, on regarde dans la table Commande s'il existe au moins une commande qui lui appartient.**
- **Si on ne trouve rien, alors NOT EXISTS est vrai.**
- **Résultat : on garde ces clients.**

Résultat :

ClientId	Nom
3	Charlie

➡️ **Charlie n'a passé aucune commande, donc il apparaît.**

🔸 **2. Même résultat avec NOT IN**

On peut aussi écrire la même chose autrement :

```
SELECT *

FROM Client cl

WHERE cl.ClientId NOT IN (

   SELECT ClientId

   FROM Commande

)

ORDER BY ClientId;
```

✅ **Cette requête fait presque la même chose :**
elle sélectionne les clients dont l'identifiant n'est pas présent dans la liste des ClientId de la table Commande.

Petite différence importante

- **NOT EXISTS est plus sûr, surtout s'il y a des valeurs NULL dans la table Commande.**
- **NOT IN peut donner un résultat faux ou vide si un NULL apparaît, car SQL ne sait pas comparer un NULL.**

Client

ClientId	Nom
Alice	Clire
Bob	Charlie

Commande

CommandeId	Id
1	1
2	NULL

7-Insertion multi ligne :

Création et insertion

```
CREATE TABLE
 NomsetHVoldesPilotes
 (nom VARCHAR(16),
   nbHVol DECIMAL(7,2),
    compa CHAR(4));

INSERT INTO NomsetHVoldesPilotes
      SELECT nom,nbHVol,compa
           FROM Pilote;
```

Requête et résultat

```
mysql> SELECT * FROM NomsetHVoldesPilotes;
+------------------+---------+-------+
| nom              | nbHVol  | compa |
+------------------+---------+-------+
| Gratien Viel     |  450.00 | AF    |
| Didier Donsez    |    0.00 | AF    |
| Richard Grin     | 1000.00 | SING  |
| Placide Fresnais | 2450.00 | CAST  |
| Daniel Vielle    |    NULL | AF    |
+------------------+---------+-------+
```

8-Limitation du nombre de lignes :

Requête

Deuxième et troisième (ordre de clé) pilote.

```
SELECT * FROM Pilote LIMIT 1,2;
```

Résultat

```
+--------+---------------+---------+-------+
| brevet | nom           | nbHVol  | compa |
+--------+---------------+---------+-------+
| PL-2   | Didier Donsez |    0.00 | AF    |
| PL-3   | Richard Grin  | 1000.00 | SING  |
+--------+---------------+---------+-------+
```

Les deux pilotes les plus expérimentés (par ordre du nombre d'heures de vol).

```
SELECT * FROM Pilote ORDER BY
nbHvol DESC LIMIT 2;
```

```
+--------+------------------+---------+-------+
| brevet | nom              | nbHVol  | compa |
+--------+------------------+---------+-------+
| PL-4   | Placide Fresnais | 2450.00 | CAST  |
| PL-3   | Richard Grin     | 1000.00 | SING  |
+--------+------------------+---------+-------+
```

9-fonctions chaines de caractères :

concat: SELECT concat('commande N°:',CommandeId, 'Client N°:',ClientId) from commande

concat('commande N°:',CommandeId, 'Client N°:',ClientId)
commande N°:10643Client N°:ALFKI
commande N°:10692Client N°:ALFKI

substring : SELECT ContactNom,SUBSTRING(ContactNom, 1,4) FROM client

ContactNom	SUBSTRING(ContactNom, 1,4)
Maria Anders	Mari

10-fonctions Dates :

***to_days(d) : permet de convertir une date en jours (pour faire des calcul)**

***SELECT DATEDIFF(DateDemandee, Datecommande) as Difference FROM `commande`**

***day(d) : extraire le jour d'une date**

***month(d) : extraire le mois d'une date**

***year(d) : extraire l'année d'une date**

***DATE_ADD(DateCommande, INTERVAL 30 DAY) as Plus30j,**

***DATE_SUB(DateCommande, INTERVAL 30 DAY) as Moins30j**

11-Fonctions de groupe :

Pilote

brevet	nom	nbHVol	prime	embauche	typeAvion	compa
PL-1	Gratien Viel	450	500	1965-02-05	A320	AF
PL-2	Didier Donsez	0		1995-05-13	A320	AF
PL-5	Daniel Vielle	400	600	1965-01-16	A340	AF
PL-6	Francoise Tort		0	2000-12-24	A340	CAST
PL-3	Richard Grin	1000		2001-09-11	A320	SING
PL-4	Placide Fresnais	2450	500	2001-09-21	A330	SING

Fonction	Exemples

AVG	Moyenne des heures de vol et des primes pour chaque compagnie.
	`SELECT compa, AVG(nbHVol), AVG(prime) FROM Pilote GROUP BY(compa) ;`
	`+-------+-------------+------------+`
	`| compa | AVG(nbHVol) | AVG(prime) |`
	`+-------+-------------+------------+`
	`| AF | 283.333333 | 550.0000 |`
	`| CAST | NULL | 0.0000 |`
	`| SING | 1725.000000 | 500.0000 |`
	`+-------+-------------+------------+`

Nombre d'heures de vol le plus élevé, date d'embauche la plus récente pour chaque compagnie.

```
SELECT compa, MAX(nbHVol), MAX(embauche) "Date+"
       FROM Pilote GROUP BY(compa) ;
+-------+-------------+------------+
| compa | MAX(nbHVol) | Date+      |
+-------+-------------+------------+
| AF    |      450.00 | 1995-05-13 |
| CAST  |        NULL | 2000-12-24 |
| SING  |     2450.00 | 2001-09-21 |
+-------+-------------+------------+
```

Plusieurs colonnes dans le `GROUP BY`	Nombre de pilotes qualifiés par type d'appareil et par compagnie. `SELECT compa,typeAvion, COUNT(brevet) FROM Pilote GROUP BY compa,typeAvion ;`

```
+-------+-----------+---------------+
| compa | typeAvion | COUNT(brevet) |
+-------+-----------+---------------+
| AF    | A320      |             2 |
| AF    | A340      |             1 |
| CAST  | A340      |             1 |
| SING  | A320      |             1 |
| SING  | A330      |             1 |
+-------+-----------+---------------+
```

Fonction	Exemples
`GROUP BY` et `HAVING`	Compagnies (et nombre de leurs pilotes) ayant plus d'un pilote. `SELECT compa, COUNT(brevet) FROM Pilote GROUP BY(compa) HAVING COUNT(brevet)>=2 ;`

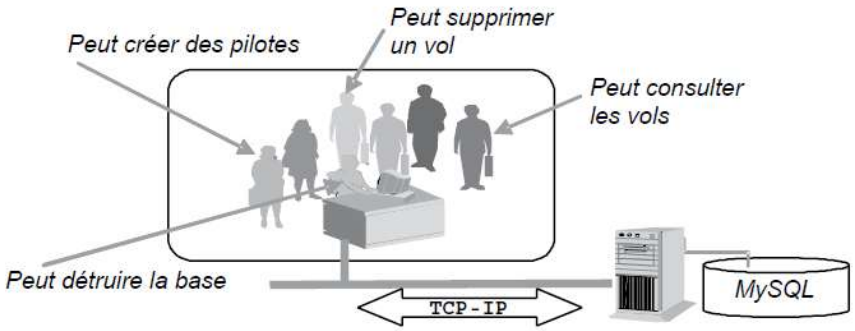
```
+-------+---------------+
| compa | COUNT(brevet) |
+-------+---------------+
| AF    |             3 |
| SING  |             2 |
+-------+---------------+
```

12-La fonction si :

if(fonction='analyste','c1',if(fonction='vendeur','c2','c3')) from emp2

13-Contrôle des données (DCL)

Comme dans tout système multi-utilisateur, l'usager d'un SGBD doit être identifié avant de pouvoir utiliser des ressources. Les accès aux informations et à la base de données doivent être contrôlés à des fins de sécurité et de cohérence. La figure suivante illustre un groupe d'utilisateurs dans lequel existe une classification entre ceux qui peuvent consulter, mettre à jour, supprimer des enregistrements.



Si quand vous arrivez sur l'index de PHPMyAdmin et qu'il ne vous demande plus de login/mot de passe et qu'il essaye de se connecter automatiquement c'est qu'il y a un souci de configuration dans le fichier (wamp\apps\phpmyadmin3.4.10.1\config.inc.php).

Il faut trouver la ligne $cfg['Servers'][$i]['auth_type'] et vérifier qu'elle soit comme elle suit:

$cfg['Servers'][$i]['auth_type'] = 'cookie';

:

1. cookie: qui place comme son nom l'indique un cookie, contenant utilisateur et mot de passe) sur votre ordinateur pour vous éviter de vous réauthentifier à chaque fois

2. config: utilise la config juste en dessous. Vous devez renseigner les champs $cfg['Servers'][$i]['user'], $cfg['Servers'][$i]['password'] et $cfg['Servers'][$i]['host']. Il se connectera alors de manière automatique.

13-1.Gestion des utilisateurs :

Création d'un utilisateur (**CREATE USER**) :

Pour pouvoir créer un utilisateur, vous devez posséder le privilège `CREATE USER` ou `INSERT` sur la base système `mysql`(car c'est la table `mysql.user`qui stockera ce nouvel arrivant).

Instruction SQL	Résultat
`CREATE USER soutou@localhost IDENTIFIED BY 'iut';`	*soutou* est déclaré « utilisateur à accès local », il devra se connecter à l'aide de son mot de passe.

Par défaut, les utilisateurs, une fois créés, n'ont aucun droit sur aucune base de données (à part en lecture écriture sur la base `test` et en lecture seule sur la base `information_schema`).

13-2.Liste des utilisateurs

À propos de `root`, on le retrouve dans la table `user` de la base `mysql`(`mysql.user`). L'extraction des colonnes `User` et `Host` restitue la liste des utilisateurs connus du serveur. Si `root`n'avait pas sélectionné la base `mysql`, la commande à exécuter aurait été « `SELECT User,Host FROM mysql.user;` ».

`SELECT User,Host FROM user;.`

13-3.Modification d'un utilisateur :

SET PASSWORD FOR 'MonNomDeUSER'@'localhost' = PASSWORD('Nouv-MotDEPasse')

13-4.Renommer un utilisateur (RENAME USER) :

Pour pouvoir renommer un utilisateur, vous devez posséder le privilège `CREATE USER` (ou le privilège `UPDATE` sur la base de données `mysql`). La syntaxe SQL est la suivante :
`RENAME USER` *`utilisateur`* `TO` *`nouveauNom`*`;`

Instruction SQL	Commentaire
`RENAME USER soutou@localhost TO christiansoutou@localhost;`	L'accès *soutou* en local est renommé *christiansoutou* en local.
`RENAME USER christiansoutou@localhost TO christiansoutou@194.53.227.12;`	L'accès *christiansoutou* en local est renommé *christiansoutou* en accès distant.
`RENAME USER christiansoutou@194.53.227.12 TO soutou@localhost;`	L'accès est renommé complètement.

14-4.Suppression d'un utilisateur (DROP USER) :

Pour pouvoir supprimer un utilisateur, vous devez posséder le privilège `CREATE USER` (ou le privilège `DELETE` sur la base de données `mysql`). La syntaxe SQL est la suivante :

```
DROP USER utilisateur [,utilisateur2 ...];
```

Pour supprimer le compte `soutou`en local, la commande à lancer est :

```
DROP USER soutou@localhost;
```

14-5.Affectation de privilèges :

Instruction faite par `root`	Explication
`GRANT CREATE ROUTINE ON bdpaul.* TO 'Paul'@'localhost';`	Privilège système *database level* : `Paul` (en accès local) peut créer ou supprimer des sous-programmes catalogués dans la base `bdpaul`.
`GRANT ALTER ROUTINE ON PROCEDURE bdpaul.sp1 TO 'Paul'@'localhost';`	Privilège système *routine level* : `Paul` peut modifier la procédure `sp1` contenue dans la base `bdpaul`.
`GRANT EXECUTE ON PROCEDURE bdpaul.sp1 TO 'Jules'@'localhost';` `GRANT EXECUTE ON FUNCTION bdpaul.sp2 TO 'Jules'@'localhost';`	Privilèges système *routine level* : `Jules` peut exécuter la procédure `sp1` et la fonction `sp2` contenues dans la base `bdpaul`.

14-6.Révocation de privilèges (REVOKE) :

La révocation d'un ou de plusieurs privilèges est réalisée par l'instruction `REVOKE`. Pour pouvoir révoquer un privilège, vous devez détenir (avoir reçu) au préalable ce même privilège avec l'option `WITH GRANT OPTION`.

Instruction faite par `root`	Explication

`REVOKE CREATE`
`ON bdpaul.*`
`FROM 'Paul'@'localhost';`
	Privilège système *database level* :
	`Paul` (en accès local) ne peut plus créer de tables dans la base `bdpaul`.

`REVOKE ALTER,INSERT,UPDATE(ISBN)`
`ON bdpaul.Livre`
`FROM 'Paul'@'localhost';`
	Privilège objet *table level* :
	`Paul` ne peut plus modifier la structure (ou les contraintes), insérer et modifier la colonne `ISBN` de la table `Livre` contenue dans la base `bdpaul`.

Tout en une fois !

Il existe une instruction qui révoque tous les droits en une fois.

```
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'Jules'@'localhost';
```

Voici le modèle relationnel des données de la base de données **AcciRoute** pour représenter les rapports d'accidents de la route. Le type des attributs est fourni sur le modèle. Les clés primaires sont soulignées et les clés étrangères sont écrites en italique :

Personne (**Cin : varchar(22)**, nom : varchar(35), villeP : varchar(50))

Voiture (**imma : varchar(12),**modele : varchar(20), annee : int, *nas* : *int*)

Accident (**dateAc : date, *nas* : int**, dommages : single, villeAc : varchar(50), *imma* : *varchar(12*))

Travail à faire :

1) Créer la base de données sous Mysql. (3pt)
2) Créer le diagramme de la base et saisir un jeu d'enregistrements dans chaque table (1pt)
3) Ajouter les utilisateurs suivants et accorder leur les permissions spécifiées (1pt)

Login	**Permissions**
said	Propriétaire de la base
laila	Lecture des champs **Cin** et nom de la table Personne
mourad	Lecture de toutes les tables et modification de la table Accident

Optimisation des bases de données relationnelles (MySQL)

Introduction

L'optimisation d'une base de données est essentielle pour garantir des performances élevées, en particulier dans des systèmes où les requêtes et les opérations de manipulation des données sont fréquentes.

Dans MySQL, plusieurs objets permettent d'améliorer l'efficacité et la maintenance de la base de données. Parmi ces outils, on retrouve les **index**, **vues**, **procédures**, **fonctions** et **triggers**. Chacun de ces objets joue un rôle spécifique pour améliorer les temps de réponse, réduire la charge serveur et automatiser certaines tâches, contribuant ainsi à une meilleure gestion et rapidité de l'accès aux données.

1-Index

Les **index** accélèrent l'accès aux données dans une table en créant des repères pour les colonnes fréquemment recherchées ou jointes.

- **Création d'un Index** :

  ```
  CREATE INDEX nom_index ON table (colonne);
  ```

- **Avantage** : Accélère les recherches (`SELECT`), tris et jointures.

  Exemple :

  Analyse des Performances avec EXPLAIN

2. Vue (View)

Une **vue** est une table virtuelle qui simplifie les requêtes complexes sans stocker les données, réduisant la redondance dans les requêtes SQL.

- **Création d'une Vue** :

  ```
  CREATE VIEW nom_vue AS
  SELECT colonne1, colonne2 FROM table WHERE condition;
  ```

- **Avantage** : Simplifie les requêtes complexes, améliore la sécurité en limitant l'accès direct aux tables.

2-1. Vues Modifiables

Définition : Une vue est considérée comme modifiable si elle permet d'insérer, de mettre à jour ou de supprimer des données dans les tables sous-jacentes via la vue.

Caractéristiques :

- Les vues modifiables peuvent être utilisées comme une interface pour modifier les données des tables d'origine sans avoir à interagir directement avec ces tables.
- Pour qu'une vue soit modifiable, elle doit répondre à certaines conditions :
    - La vue doit être basée sur une seule table.
    - La vue ne doit pas contenir d'agrégations (comme `SUM`, `COUNT`, etc.).
    - La vue ne doit pas avoir de clauses `DISTINCT`, `GROUP BY`, `HAVING` ou `ORDER BY`.
    - Toutes les colonnes dans la vue doivent correspondre aux colonnes dans la table sous-jacente, sans transformation ni calcul.

- **Exemple de Vue Modifiable** :

```
CREATE VIEW vue_employes AS
SELECT id, nom, salaire FROM employes;

-- Mise à jour via la vue
UPDATE vue_employes SET salaire = 5000 WHERE id = 1;
```

2-2. Vues Non Modifiables

Définition : Une vue est non modifiable si elle ne permet pas de modifier les données des tables sous-jacentes via la vue.

Caractéristiques :

- Les vues non modifiables sont souvent utilisées pour des raisons de sécurité ou de simplification des requêtes.
- Elles peuvent inclure des jointures, des agrégations, des sous-requêtes ou d'autres éléments qui rendent la modification des données impossible.
- Même si elles affichent des données de plusieurs tables, il n'est pas possible d'utiliser des instructions `INSERT`, `UPDATE` ou `DELETE` directement sur la vue.

Exemple de Vue Non Modifiable :

```
CREATE VIEW vue_salaire_total AS
SELECT departement_id, SUM(salaire) AS total_salaire
FROM employes
GROUP BY departement_id;


-- Tentative de mise à jour (ne fonctionnera pas)
UPDATE vue_salaire_total SET total_salaire = 10000 WHERE
departement_id = 1; -- Cela génère une erreur
```

3. Procédure Stockée (Stored Procedure)

Une **procédure** stockée est un ensemble d'instructions SQL exécutées comme un bloc, utile pour les tâches répétitives et la manipulation de données en masse.

Avantage : Regroupe plusieurs requêtes, réduit les échanges réseau.

4. Fonction Stockée (Stored Function)

Les **fonctions** stockées sont similaires aux procédures mais retournent une valeur. Elles sont idéales pour les calculs et transformations de données.

Avantage : Peut être utilisée dans une requête SELECT pour les calculs.

5. Trigger

Les **triggers** (déclencheurs) sont des blocs de code qui s'exécutent automatiquement lors d'opérations spécifiques (`INSERT`, `UPDATE`, `DELETE`), utiles pour automatiser des actions.

Avantage: Automatisation d'actions complexes (ex. : journalisation, vérifications de données).

Exporter une base de données :

mysqldump -u root -p ma_base_de_donnees > /chemin/vers/dossier/backup.sql

Importer une base de données :

mysql -u [nom_utilisateur] -p [nom_base_de_donnees] < [chemin/fichier_de_sortie.sql]

Récapitulatif global

Objet	Rôle principal	Impact sur l'optimisation
Index	Accélère la recherche et les jointures	Améliore fortement les performances
Vue	Simplifie l'accès aux données et renforce la sécurité	Facilite la maintenance, peu d'impact direct sur la vitesse
Procédure / Fonction	Automatise les traitements complexes côté serveur	Réduit la charge client et les échanges réseau
Trigger	Automatisation lors d'événements (INSERT, UPDATE, DELETE)	Maintient l'intégrité et réduit les traitements manuels

